Protocols, cryptanalysis and mathematical cryptology (INFO-F514)

“Introduction - e-voting”

Liran Lerman and Christophe Petit

Université libre de Bruxelles

What this course is about

- Modern cryptology and cryptanalysis tools;
  design, analysis and implementation
  of cryptographic primitives and protocols
- Critical analyzis of some current issues in the field
  $\rightarrow$ e-voting this year
- Research topics and methods in cryptology

Outline

Course organization

Introduction to e-voting

Conclusion

Outline

Course organization

Introduction to e-voting

Conclusion

Lecturers

Christophe Petit

- E-mail: christophe.petit@ulb.be
- Background on mathematical cryptography
- Presenting first part of the course

Liran Lerman

- E-mail: liran.lerman@ulb.be
- Background on applied cryptography
- Presenting second part of the course

Invited lectures by industry experts

Learning approaches

- Regular lectures (either on-campus or online)
- Your own preparation to lectures
- Your own (guided) reading of research papers (with feedback from us upon request)
- Project

Lecture content

Week	Topic
1	Introduction to the course and e-voting
2	"Provable security"
3	Homomorphic encryption
4	Zero-knowledge proofs
5	Cryptanalysis
6	Post-quantum cryptography
7	Differential fault analysis
8	Security of AI
9	Security/Crypto in industry (Part 1)
10	Security/Crypto in industry (Part 2)
11	Threat model
12	Secure software implementation

Lectures: organization

Before each lecture

- Read associated research paper and answer related questions
- You are welcome (but not requested) to submit your answers; we will provide feedback on them;
  they do not count towards the final mark
- Purpose: prepare for course and evaluation

During the lectures

- We will go over the slides quickly, assuming you have already discovered their content and tried to understand it
- We will answer any questions you will ask

Evaluation: oral exam

- 50% on lecture content
- 50% on group project

(! this has changed from last year...)

Group project: purposes

Expand your cryptography knowledge
on a topic of your choice
beyond what is covered in the lectures

+ a glance at cryptography research

Group project main steps

- Form a small group
- Choose a research problem and paper
- Submit a project proposal on UV; get feedback
- Study the paper
- Submit a draft group report on your findings; get feedback
- Submit a group report
- Submit an individual report
- Do an oral examination

Deadlines

February 27, 2026: Build your group and choose a project

- Form your own group
- Five students per group
- Identify your project topic
- Identify at least one research paper related to crypto
- Submit a one-page description of your project by the deadline on UV

Deadlines

March 27, 2026: Progress report: brief statement on

- Progress made so far on the project
- Workload distribution
- Difficulties encountered so far

Deadlines

April 30, 2026: Submit group report including:

- At most 10 pages (including references and appendices)
- Introduction motivating/defining the problem
- Description of the paper's main contributions
- Explanation of their assumptions/ methodology/ ideas/ results
- When appropriate: description of your own attempts at reproducing their findings
- Explain why the results are important
- Explain what are their current limitations
- Possibly compare the paper to other approaches

Deadlines

May 8th, 2026: Submit individual report including:

- At most 4 pages
- Summarize the project (3 pages)
- Explain how the work was distributed among students (1 page)

Deadlines

June exam period: oral examination

- Individual examination on the project you worked on (+ course content)
- You may take your group project report with you

Course mark

- Course mark = 0.5 project + 0.5 lecture content
- Oral examination on both parts
- We expect that all group members contribute equally to the project (quantitatively and qualitatively)
- We will assess this based on individual reports and interactions with you (**get in touch early if issues!**)
- Project mark will depend on the quality of group and individual reports, on individual contributions to project and on individual oral examination

Our expectations

- Prepare the lectures beforehand
- Be curious
- Be critical of references you read (including our own lectures)
- Participate fairly to the project
- Don't be shy in asking questions !

The only dumb question is the one you're afraid to ask...
until it appears at the exam

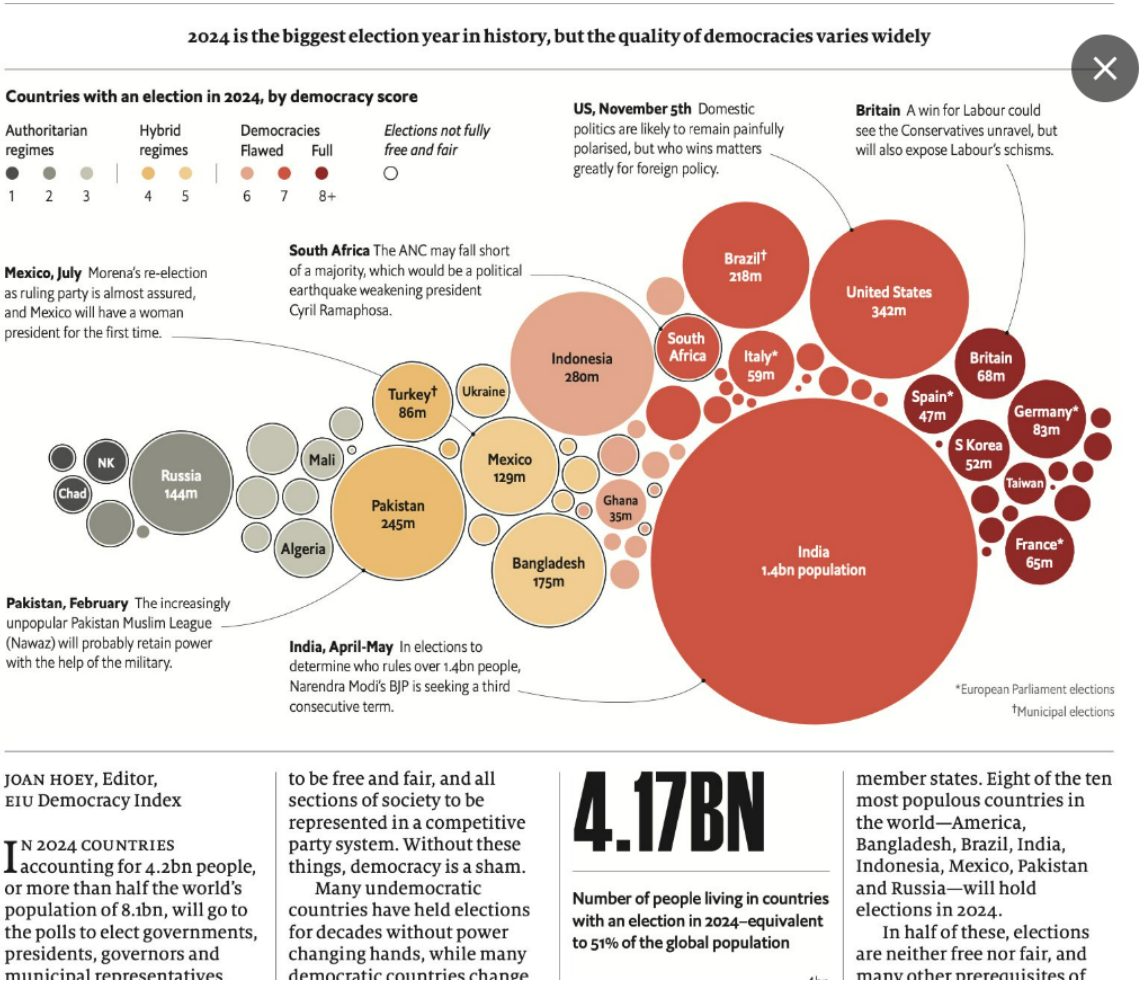
Outline

Course organization

Introduction to e-voting

Conclusion

World elections (in 2024)



JOAN HOEY, Editor, EIU Democracy Index

IN 2024 COUNTRIES accounting for 4.2bn people, or more than half the world's population of 8.1bn, will go to the polls to elect governments, presidents, governors and municipal representatives to be free and fair, and all sections of society to be represented in a competitive party system. Without these things, democracy is a sham.

Many undemocratic countries have held elections for decades without power changing hands, while many democratic countries change

4.17BN

Number of people living in countries with an election in 2024–equivalent to 51% of the global population

member states. Eight of the ten most populous countries in the world—America, Bangladesh, Brazil, India, Indonesia, Mexico, Pakistan and Russia—will hold elections in 2024.

In half of these, elections are neither free nor fair, and many other prerequisites of

Picture credit:www.economist.com

At ULB

Dear Members of the University Community,

For the first time in the history of our University, all members of the University community will have the opportunity to vote in the Rectoral Election.

Whether you are a member of the Academic or Scientific Body, a student or a member of the Administrative staff, you are invited to participate, **on 15 and 16 May 2024**, in the election of the person who will become Rector at the start of the 24-25 academic year for a four-year term.

This election will take place entirely online, through **electronic voting**.

All the information regarding this significant election for the University can be found on ulb.be/elections (in French).

The **list of candidates** will be published on this same page on the evening of 22 April 2024.

You can also check on the website **whether you need to register before voting**.

This is a crucial election for the University. It is important that you participate in large numbers.

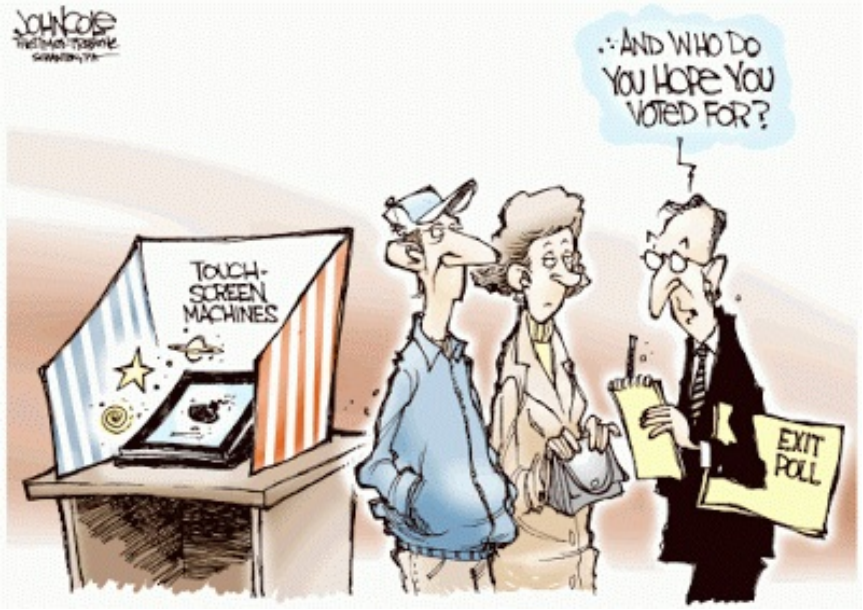
Yours sincerely,

Monique Tavernier,
Head of the Registrar's Office

Bernard De Cannière,
President of the Board

Electronic elections

In the News: Michigan Voters with Blindness Prefer ES&S



Kids at hacking conference show how easily US elections could be sabotaged

Headlines from: Election Systems & Software, retrieved 15/11/2018; The Guardian, 22/08/2018.
Cartoon by John Cole published on `e-lected.blogspot.com`. Picture of the Diebold AccuVote-TS.

At ULB

From Belenios ULB elections server <noreply@ulb.be>

To Christophe Petit

Subject **Your vote for election Corps académique à l'Assemblée plénière de l'Université**

```
Dear chripeti,

Your vote for election

  Corps académique à l'Assemblée plénière de l'Université

has been recorded. Your smart ballot tracker is

  JpoITgnnZ1ilqAl6h98C2f6gYOUS9e5ZFOz6Qk8Bx6A

You can check its presence in the ballot box, accessible at
  https://belenios.ulb.be/elections/chiWrdxyw6sdrD/ballots

Results will be published on the election page
  https://belenios.ulb.be/elections/chiWrdxyw6sdrD/

To get more information, please contact:
  elections@ulb.be

--
Belenios
From - Mon Jan 27 15:24:19 2025
```

Outline

Course organization

Introduction to e-voting
- An overview of "e-voting systems"
- Homomorphic encryption approach
- Mixnet approach
- Practical case study: Helios

Conclusion

Outline

Course organization

Introduction to e-voting
- An overview of "e-voting systems"
- Homomorphic encryption approach
- Mixnet approach
- Practical case study: Helios

Conclusion

Direct Record Electronic (DRE) machines

- Used for real elections in the USA since 1974
- Also used in Australia, Brazil, Canada, India, ...
- No cryptography involved
- General purpose computer with election software and some user interface, located at the polling station
- Designs often kept secret and claimed secure ...

DRE "security" record

- Designs often kept secret and claimed secure but
  - Code had common software vulnerabilities
  - Issues with software update mechanisms
  - Hardcoded cryptographic keys
  - Malware could reveal votes, modify them and propagate
- Further security issues in development and use
  - Code leaked on public FTP server
  - Weak physical locks to protect machine
  - Discarded machines still had votes recorded

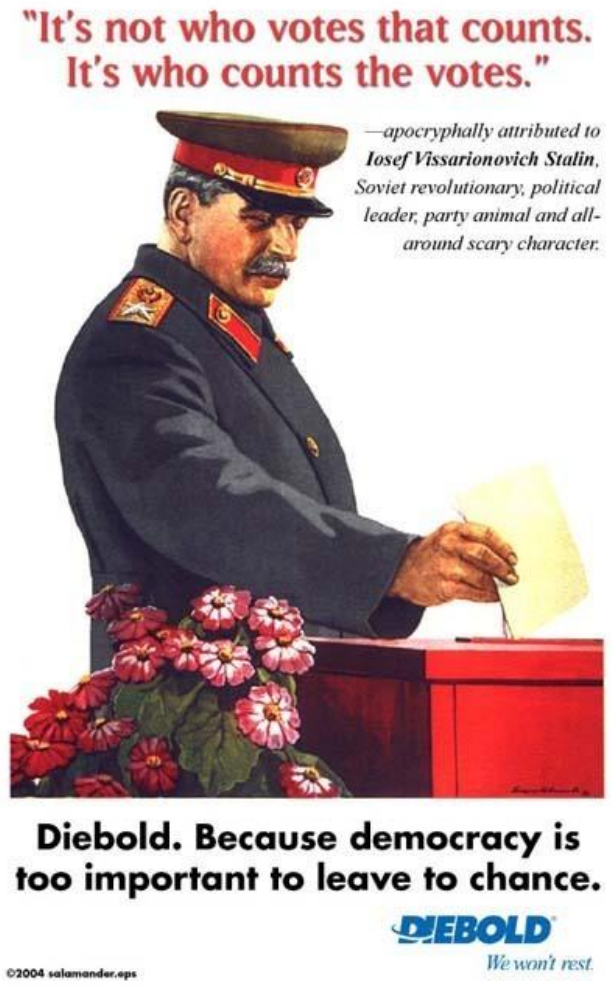
DRE questionned

- DRE machines particularly criticised after US 2000 presidential elections (Gore vs Bush)
- 2009, German Federal Constitutional Court:

  when using voting machines the "verification of the result must be possible by the citizen reliably and without any specialist knowledge of the subject"

- e-voting abandonned in the Netherlands
- Still used in Brazil, India, DR Congo, ...

DRE questionned



https://columbusfreepress.com/article/diebold-indicted-its-spectre-still-haunts-ohio-elections

Cryptography to the rescue

- Legislation passed to fund research on e-voting
- Cryptographers designed **end-to-end verifiable** voting systems, where voters can verify that their votes are
  - Cast as intended
  - Recorded as cast
  - Counted as recorded
- Now used for general elections in Estonia and more widely for elections with low coercion threat level
- This course only about those systems

e-voting = no paper?

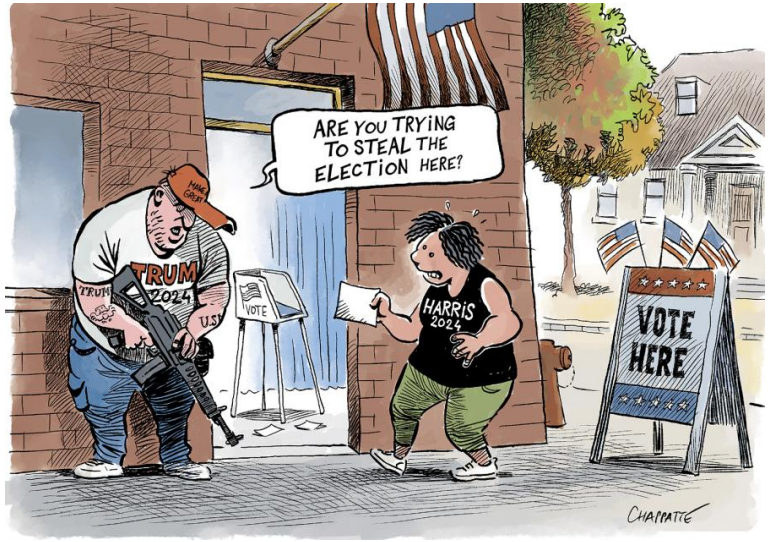
- Many interesting "e-voting schemes" (developed by cryptographers) involve paper, scan codes, punch cards, ...
- This course not about those systems; we will focus on purely electronic techniques
- Ask us for references if interested

A small exercice

- What are the main security features expected from an e-voting system?
- Reflect on the voting systems you have used in the past. Did they satisfy those security properties?

Main security goals

- Correctness
  - Result consistent with intended votes
  - Verifiability: system should give correctness guarantees
- Privacy
  - Confidentiality of individual votes
  - Coercion resistance: cannot prove your vote



picture credit: `https://www.chappatte.com/en/images/already-contested-us-election`

Challenges

- Fundamental tension between verifiability and privacy: how to certify the results without seeing the votes?
- Additional challenges with careless or malicious users, malicious software and hardware components

Reminder: encryption

- Symmetric/ public key encryption schemes
  - Key generation algorithm: takes security parameter, generates a secret key
  - Encryption algorithm: takes plaintext and (public) key, returns ciphertext
  - Decryption algorithm: takes ciphertext and secret key, returns plaintext
- Security properties: IND-CPA, IND-CCA
- Use this to ensure confidentiality of individual votes?

e-voting attempt

- Election officials generate public key encryption keys
- Each voter encrypts their vote with the public key; vote is sent to election officials
- Each vote is decrypted with secret key and tallied
- Issues?
  - Election officials can know all the votes
  - No guarantee they do not change, add or remove votes

Improvement: publishing encrypted votes

- Encrypted votes published on a bulletin board
- Voters can verify their vote is on the bulletin board
- Tallying process can be witnessed to ensure no vote is added/changed compared to bulletin board
- Each vote can also be signed by voter; in which case everybody can check their integrity on the bulletin board (but not possible deletions)
- Remaining issues?
  - Election officials and witnesses can know all the votes
  - Users can prove how they voted

Two approaches to e-voting

- Homomorphic encryption: use encryption scheme such that
$$Enc(v_1 + v_2) = Enc(v_1) \otimes Enc(v_2)$$
for some operation $\otimes$,
  $\rightarrow$ Only end result needs to be decrypted

- Mixnets: shuffle the votes before decryption
  $\rightarrow$ Votes can no longer be linked to voters
  $\rightarrow$ Votes can be decrypted individually

Crypto tools needed

- Homomorphic encryption
  - To allow vote aggregation, ciphertext re-randomization
  - ElGamal encryption is homomorphic !
- Zero-knowledge protocols: prove some statement about secret values without revealing anything else about them
  - To prove that encrypted votes contain legitimate values, that election result is computed and decrypted correctly, that shuffle is correctly done, ...
- Secret sharing, treshold encryption
  - To prevent single authority from reading individual votes
  - Can also be done with ElGamal

Outline

Course organization

Introduction to e-voting
- An overview of "e-voting systems"
- Homomorphic encryption approach
- Mixnet approach
- Practical case study: Helios

Conclusion

ElGamal encryption

- Public key is cyclic group G, generator g and $h = g^s$
- Private key is s
- Encryption of $m \in G$ is $(c_1, c_2) = (g^r, mh^r)$ for some random r
- To decrypt (c_1, c_2) compute $c_2/c_1^s = m$

ElGamal: many useful properties

- Homomorphic: compute encryption of election result from encryption of individual votes
- Re-randomizable: modify ciphertext without changing underlying plaintext
- Treshold decryption: make sure decryption only possible when trustees work together

Homomorphic properties of ElGamal

- Let $(c_1, c_2) = (g^r, mh^r)$ encryption of m
- Let $(c_1', c_2') = (g^{r'}, m'h^{r'})$ encryption of m'
- Then

$$(C_1, C_2) := (c_1 c_1', c_2 c_2') = (g^{r+r'}, mm'h^{r+r'})$$

  is an encryption of mm'

- If $m' = 1$ then (C_1, C_2) is a new encryption of m
  $\Rightarrow$ can re-randomize ciphertext without knowing plaintext

Exponential ElGamal

- Public key is cyclic group G, generator g and $h = g^s$, and a bound B; private key is s
- Encryption of $0 \leq m < B$ is $(c_1, c_2) = (g^r, g^m h^r)$ for some random r
- To decrypt (c_1, c_2) first compute $c_2/c_1^s = g^m$, then solve DLP over a small interval to get m
- DLP feasible over small intervals (exhaustive search, birthday algorithms)
- Same security as ElGamal (prove it!)

Homomorphic properties of Exponential ElGamal

- Let $(c_1, c_2) = (g^r, g^m h^r)$ encryption of m
- Let $(c_1', c_2') = (g^{r'}, g^{m'} h^{r'})$ encryption of m'
- Then

$$(C_1, C_2) = (c_1 c_1', c_2 c_2') = (g^{r+r'}, g^{m+m'} h^{r+r'})$$

  is an encryption of $m + m'$ (modulo group order)

- If $m' = 0$ then (C_1, C_2) is a new encryption of m
  $\Rightarrow$ can re-randomize ciphertext without knowing plaintext

e-voting attempt

- For each candidate, election officials generate keys for Exponential ElGamal encryption
- Voters encrypt 1 if they want to vote for a candidate; they encrypt 0 otherwise
- Votes are published
- Encryption of result can be computed by anyone
- Election officials decrypt result

- Issues?
  - No guarantee that votes are well-formed (e.g. encryption of 1000 or -1)
  - Election officials can also decrypt individual votes
  - Users can prove how they voted

Secret sharing

- Goal: decryption key shared among several trustees
- ElGamal variant:
  - Each trustee chooses secret s_i and shares $h_i = g^{s_i}$
  - Public key is $h = \prod_i h_i = g^{\sum_i s_i}$
  - Encryption of m like in ElGamal: $(c_1, c_2) = (g^r, mh^r)$ for some random r
  - To decrypt (c_1, c_2) trustees must jointly compute $c_2 / \prod_i c_1^{s_i} = m$

Fancy variant: treshold encryption

- Goal: decryption key shared among several trustees
- Moreover, there is an integer t such that any set of $n \geq t$ trustees can decrypt, but no smaller set can
- Assume each trustee has a private key share $s_i = F(i)$ for some polynomial F known to none, with $\deg F = k - 1$
- Corresponding secret key is $F(0)$, public key is $h = g^{F(0)}$
- Encryption of m like in ElGamal: $(c_1, c_2) = (g^r, mh^r)$ for some random r
- To decrypt (c_1, c_2) trustees need compute $c_2/c_1^{F(0)} = m$

Treshold ElGamal: Lagrange interpolation

- For any subset $\{x_1, \ldots, x_t\} \in \{1, \ldots, n\}$, we have

$$F(0) = \sum_i L_i F(x_i)$$

where

$$L_i = \prod_{j \neq i} \left(\frac{-x_j}{x_i - x_j} \right)$$

- Proof: use Lagrange interpolation $F(x) = \sum_i L_i(x) F(x_i)$ where

$$L_i(x) = \prod_{j \neq i} \left(\frac{x - x_j}{x_i - x_j} \right)$$

Treshold ElGamal: public key and decryption

- Public key computed as

$$\prod_i \left(g^{F(x_i)}\right)^{L_i} = g^{F(0)}$$

- Decryption performed as

$$c_2 / \prod_i \left(c_1^{F(x_i)}\right)^{L_i} = c_2 / c_1^{F(0)} = m$$

Treshold ElGamal: secret shares

- Polynomial F should remain secret from trustees: indeed secret key is $F(0)$
- How to make sure party i has $F(i)$?
- Solution (Pedersen '98)
  - Each party chooses a random polynomial $F_i(x)$
  - Party i privately sends $F_i(j)$ to party j
  - Define $F(x) = \sum F_i(x)$
  - So $F(j) = \sum F_i(j)$
- Note: $F(x)$ random as long as one trustee is honest

e-voting attempt

- For each candidate, election officials generate keys for treshold exponential ElGamal encryption
- Voters encrypt 1 if they want to vote for a candidate; they encrypt 0 otherwise
- Votes are published
- Encryption of result can be computed by anyone
- Election officials must cooperate to decrypt result; we assume they will not cooperate to decrypt individual votes
- Issues?
  - No guarantee that votes are well-formed (e.g. encryption of 1000 or -1)
  - Users can prove how they voted

Encryption well-formed

- Must prevent voters from encrypting 1000, -1, ...
- A vote is well-formed if it is an exponential ElGamal encryption of either 0 or 1, namely

$$(c_1, c_2) = (g^r, g^m h^r)$$

  where $m \in \{0, 1\}$
- In other words: either one of

$$(g, h, c_1, c_2), \qquad (g, h, c_1, c_2/g)$$

  is a Diffie-Hellman tuple
- Solution: add a zero-knowledge proof of this!

Encryption well-formed: interactive proof

- Combine Chaum-Pedersen and OR proofs
- The protocol will include
  - Using witness r, an actual Chaum-Pedersen proof that $(g, h, c_1, c_2/g^m)$ is a Diffie-Hellman tuple,
  - A simulated Chaum-Pedersen proof that $(g, h, c_1, c_2/g^{1-m})$ is a Diffie-Hellman tuple (note that it is in fact not a Diffie-Hellman tuple)

OR proof

Prover(x_0, x_1, w_b)		Verifier(x_0, x_1)
$\alpha_b \leftarrow P_b(x_b, w_b)$; Choose random β_{1-b}; $(\alpha_{1-b}, \beta_{1-b}, \gamma_{1-b}) \leftarrow S_{1-b}(x_{1-b}, \beta_{1-b})$;		
	$\xrightarrow{\alpha_0, \alpha_1}$	
		$\beta_b \leftarrow V_b(x_b, \alpha_b)$;
	$\xleftarrow{\beta}$	
$\beta_b = \beta \oplus \beta_{1-b}$; $\gamma_b \leftarrow P_b(x_b, w_b, \alpha_b, \beta_b)$;		
	$\xrightarrow{\beta_0, \beta_1, \gamma_0, \gamma_1}$	
		Accept if $\beta_0 \oplus \beta_1 = \beta$ and $V_0(\alpha_0, \beta_0, \gamma_0) = 1$ and $V_1(\alpha_1, \beta_1, \gamma_1) = 1$

Chaum-Pedersen protocol

- Prover knows x such that $h_1 = g_1^x$ and $h_2 = g_2^x$
- Prover chooses random r
- Prover sends $R_1 = g_1^r$ and $R_2 = g_2^r$ to verifier
- Verifier sends a random challenge c
- Prover answers with $s = r + cx$
- Verifier checks $g_1^s = R_1 h_1^c$ and $g_2^s = R_2 h_2^c$

Encryption well-formed: interactive proof

- Combine Chaum-Pedersen and OR proofs

Prover(g, h, c_1, c_2, r, m)		Verifier(g, h, c_1, c_2)
Choose random t; $R_{1,m} \leftarrow g^t$; $R_{2,m} \leftarrow h^t$ Choose random β_{1-m} and γ_{1-m}; $R_{1,1-m} \leftarrow g^{\gamma_{1-m}} c_1^{-\beta_{1-m}}$; $R_{2,1-m} \leftarrow h^{\gamma_{1-m}} (c_2/g^{1-m})^{-\beta_{1-m}}$;		
	$\xrightarrow{R_{1,0}, R_{2,0}, R_{1,1}, R_{2,1}}$	
		Choose random β;
	$\xleftarrow{\beta}$	
$\beta_m = \beta \oplus \beta_{1-m}$; $\gamma_m \leftarrow t + \beta_m r$;		
	$\xrightarrow{\beta_0, \beta_1, \gamma_0, \gamma_1}$	
		Accept if $\beta_0 \oplus \beta_1 = \beta$ and $g^{\gamma_0} = R_{1,0} c_1^{\beta_0}$ and $h^{\gamma_0} = R_{2,0} c_2^{\beta_0}$ and $g^{\gamma_1} = R_{1,1} c_1^{\beta_1}$ and $h^{\gamma_1} = R_{2,1} (c_2/g)^{\beta_1}$

Encryption well-formed: non-interactive proof

- Use Fiat-Shamir heuristic: proof is

$$\pi = (R_{1,0}, R_{2,0}, R_{1,1}, R_{2,1}, \beta_0, \beta_1, \gamma_0, \gamma_1)$$

where all values are computed as above,
and $\beta = H(R_{1,0}, R_{2,0}, R_{1,1}, R_{2,1}) = \beta_0 + \beta_1$

e-voting attempt

- For each candidate, election officials generate keys for treshold exponential ElGamal encryption
- Voters encrypt 1 if they want to vote for a candidate; they encrypt 0 otherwise; they append a proof of this
- Votes are published
- Correctness of votes can be checked by anyone
- Encryption of result can be computed by anyone
- Election officials must cooperate to decrypt result; we assume they will not cooperate to decrypt individual votes
- Issues?
  - Users can prove how they voted

Bonus: reducing the trust in trustees

- So far we are trusting that election officials
  - cooperate to produce the correct decryption
  - do not cooperate to decrypt individual votes
- Can we check that result is correct?
- Solution: zero-knowledge proof!
  - Election trustees jointly decrypt result $r = \sum_i m_i$
  - They jointly produce a proof of the result, namely a proof that $(g, h, c_1, c_2/g^r)$ is a valid Diffie-Hellman tuple

Reminder: Chaum-Pedersen protocol

- Prover knows x such that $h_1 = g_1^x$ and $h_2 = g_2^x$
- Prover chooses random r
- Prover sends $R_1 = g_1^r$ and $R_2 = g_2^r$ to verifier
- Verifier sends a random challenge c
- Prover answers with $s = r + cx$
- Verifier checks $g_1^s = R_1 h_1^c$ and $g_2^s = R_2 h_2^c$
- Issue here: x not known to the trustees (they individually know a share of x)

Decryption correct (non treshold variant)

- Trustee i has secret key x_i with public key g^{x_i}
- Overal public key is $\prod_i g^{x_i} = g^x$, where $x = \sum_i x_i$
- Trustee i reveals $c_1^{x_i}$ and proves that $(g, g^{x_i}, c_1, c_1^{x_i})$ is a valid Diffie-Hellman tuple using Chaum-Pedersen
- Everyone can check those proofs and then verify that

$$c_2 / \prod_i c_1^{x_i} = c_2 / c_1^x = m$$

- Non-interactive proof: use Fiat-Shamir heuristic

Decryption correct (treshold variant)

- Trustee i has secret key $x_i = F(i)$ with public key g^{x_i}
- Trustee i reveals $c_1^{x_i}$ and proves that $(g, g^{x_i}, c_1, c_1^{x_i})$ is a valid Diffie-Hellman tuple using Chaum-Pedersen
- Everyone can check those proofs and then verify that

$$c_2 / \prod_{i \in I} \left(c_1^{F(i)}\right)^{L_i} = c_2 / c_1^{F(0)} = m$$

- Non-interactive proof: use Fiat-Shamir heuristic

e-voting attempt

- For each candidate, election officials generate keys for treshold exponential ElGamal encryption
- Voters encrypt 1 if they want to vote for a candidate; they encrypt 0 otherwise; they append a proof of this
- Votes are published
- Correctness of votes can be checked by anyone
- Encryption of result can be computed by anyone
- Election officials must cooperate to decrypt result and produce a proof of correctness (we assume they will not cooperate to decrypt individual votes)
- Issues?
  - Users can prove how they voted

Outline

Course organization

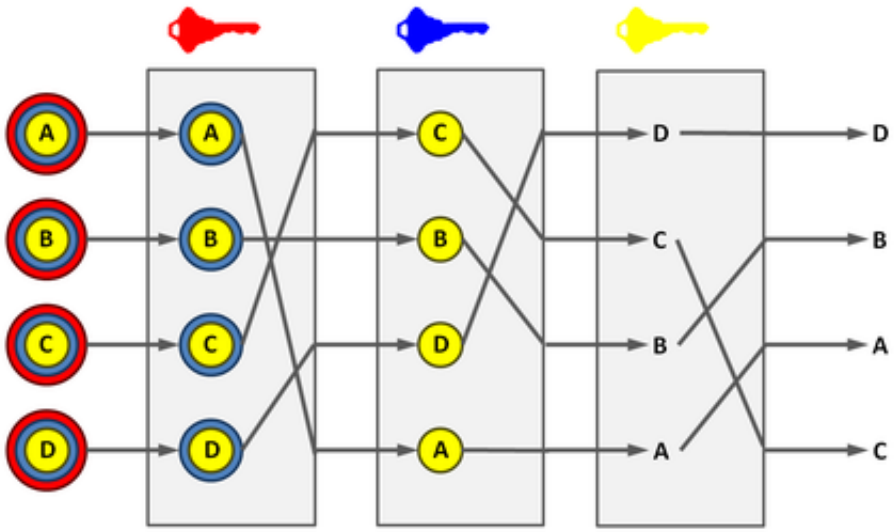
Introduction to e-voting
- An overview of "e-voting systems"
- Homomorphic encryption approach
- Mixnet approach
- Practical case study: Helios

Conclusion

e-voting attempt

- Election officials generate encryption keys
- Voters send their encrypted votes
- Trustee takes all encrypted votes, re-randomizes them and shuffles them
- Election official takes shuffled votes, decrypts, computes result, proves result
- Issues?
  - Trustee could remove/add some votes
  - Trustee and election official together can decrypt all votes

Mixnets: using multiple trustees



- Each trustee generates their keys
- Each vote is encrypted several times, one per trustee
- Each trustee removes one encryption layer and shuffles all the votes

Picture credit: Wikipedia

Optimization: ElGamal with shared secret

- ElGamal variant:
  - Each trustee chooses secret s_i and shares $h_i = g^{s_i}$
  - Public key is $h = \prod_i h_i = g^{\sum_i s_i}$
  - Encryption of m like in ElGamal: $(c_1, c_2) = (g^r, mh^r)$ for some random r
  - Decryption of (c_1, c_2) is $c_2 / \prod_i c_1^{s_i} = m$
- Single encryption layer, yet multiple decryption layers!

e-voting attempt

- Election officials jointly generate ElGamal encryption key
- Voters publish their encrypted votes
- Each election official in turn removes their part of the secret key

$$(c_1, c_2) \rightarrow (c_1, c_2/c_1^{s_i})$$

  and shuffles all the votes

- Issues?
  - c_1 part common in each decryption layer, so votes can still be tracked back to voters by last election officer
  - Election officers may cheat with decryption and shuffle

Public re-randomization

- Given a valid (exponential) ElGamal ciphertext, can produce another valid ciphertext for the same plaintext
- Use homomorphic properties
  - Let $(c_1, c_2) = (g^r, mh^r)$ encryption of m
  - Let $(c_1', c_2') = (g^{r'}, m'h^{r'})$ encryption of m'
  - Then

$$(C_1, C_2) = (c_1 c_1', c_2 c_2') = (g^{r+r'}, mm'h^{r+r'})$$

    is an encryption of mm'
  - Take $m' = 1$: then (c_1, c_2) and $(C_1, C_2) = (c_1 g^{r'}, c_2 h^{r'})$ encrypt the same message

e-voting attempt

- Election officials jointly generate ElGamal encryption key
- Voters publish their encrypted votes
- Each election official in turn removes their part of the secret key, rerandomizes the ciphertext

$$(c_1, c_2) \rightarrow (c_1, c_2/c_1^{s_i}) \rightarrow (c_1 g^r, (c_2/c_1^{s_i}) h_i^r)$$

  and shuffles all the votes (here $h_i = \prod_{j=i+1}^{N} g^{s_j}$)
- Issues?
  - Election officers may cheat with decryption and shuffle

Zero-knowledge proofs

- Each trustee must prove
  - Decryption is done properly
  - Shuffle is done properly
- Could use generic proofs for arbitrary statements, but of course dedicated proofs are far more efficient (especially when using ElGamal encryption)
- Proofs are published and can be checked by anyone

Zero-knowledge proofs

- Sako-Killian's proof that two sets of votes V_1 and V_2 are equivalent up to rerandomization and shuffling
  - Produce another equivalent set V_3
  - Receive a challenge bit
  - Depending on bit value respond with permutation and re-randomization values either between V_1 and V_3 or between V_2 and V_3
  - Repeat and use Fiat-Shamir heuristic
- Bayer-Groth, Eurocrypt 2012

Outline

Course organization

Introduction to e-voting
- An overview of "e-voting systems"
- Homomorphic encryption approach
- Mixnet approach
- Practical case study: Helios

Conclusion

Why Helios?

helios
Trust the vote.

- One of the first open source implementations of an end-to-end verifiable e-voting system
- First such tool used at "large scale" (25,000 electors) for UCLouvain 2009 rector elections (CP was there)
- Used by the IACR for their elections
- Uses many tools described above

Goals

- From theory to practice
- Full voting system, including voter registration, voting process, bulletin board, tallying
- Web-based, for easy access
- Open audit: election outcome can be verified even when the whole system is corrupt
- Acknowledged limits:
  - Only suitable for elections with low coercion risk (web-based)
  - Vote privacy relies on server goodwill
    "Trust no one for integrity, trust Helios for privacy"

Helios: IACR elections

Helios Voting Booth [exit]

IACR 2018 election

To cast a vote, you will be led through the following steps.
If you have not yet logged in, you will be asked to do so at the very end of the process.

1. **Select** your preferred options.
   You can easily navigate forwards and backwards.

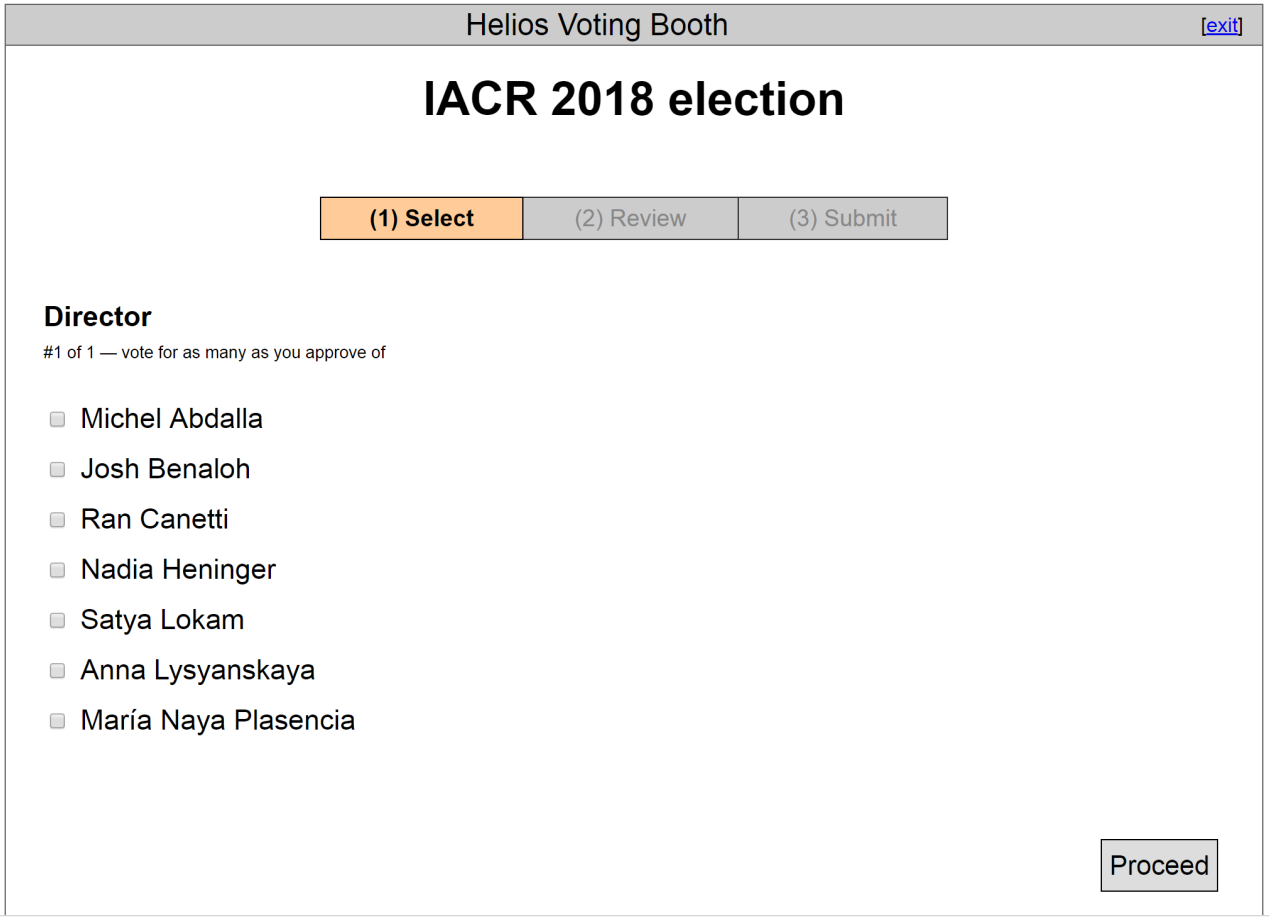
2. **Review & Confirm** your choices.
   Your choices are encrypted safely inside your browser, and you get a smart ballot tracker.

3. **Submit** your encrypted ballot.
   You will be asked to log in to submit your encrypted ballot for tallying.

Start

Election Fingerprint: `Q2gj8sqN6u+Q9dXyFRRaLtuRU6aUb4ogjNwix0GMVjU` help!

Helios: IACR elections



Helios: IACR elections

Helios Voting Booth [exit]

IACR 2018 election

(1) Select | **(2) Review** | (3) Submit

Helios is now encrypting your ballot

(0%)

This may take up to two minutes.

Election Fingerprint: `Q2gj8sqN6u+Q9dXyFRRaLtuRU6aUb4ogjNwix0GMVjU` help!

Helios: IACR elections

Helios Voting Booth [exit]

IACR 2018 election

(1) Select | **(2) Review** | (3) Submit

Review your Ballot

Audit [optional]

Question #1: Director
✓ Michel Abdalla
✓ Nadia Heninger
✓ Anna Lysyanskaya
[edit responses]

Your ballot tracker is `mLmKyrdPdxiZdTIKTgJLqlVdVrxfZDIrx3NqJ4E3SRA`, and you can print it.

Once you click "Submit", the unencrypted version of your ballot will be destroyed, and only the encrypted version will remain. The encrypted version will be submitted to the Helios server.

Submit this Vote!

Election Fingerprint: `Q2gj8sqN6u+Q9dXyFRRaLtuRU6aUb4ogjNwix0GMVjU` help!

Helios: IACR elections

Helios Voting | About | Code | Docs | FAQ | Privacy | Help!

Submitting your vote for **IACR 2018 election**...

Please provide the voter ID and password you received by email.

Voter ID:

Password:

You may cast as many ballots as you wish:
only the last one counts.

cast ballot

Your smart ballot tracker is:

`mLmKyrdPdxiZdTIKTgJLqlVdVrxfZDIrx3NqJ4E3SRA`

not logged in. log in

Helios: IACR elections

Helios Voting | About | Code | Docs | FAQ | Privacy | Help!

IACR 2018 election — Vote Successfully

Congratulations, your vote has been **successfully cast**!

Your smart ballot tracker is:

mLmKyrdPdxiZdTIKTgJLqlVdVrxfZDIrx3NqJ4E3SRA

[return to election info]

not logged in. log in

Helios 1.0

- Paper and implementation by Ben Adida (Usenix 2008)
- Mixnet approach with a single trustee (Helios)
- ElGamal encryption
- Hash of encryption given to voters as receipt
- Bulletin board of votes
- Sako-Killian proofs for mixnets
- Chaum-Pedersen proofs for correct decryptions
- Ballot auditing
- "Coerce me!" button

Auditing ballots

- Votes are encrypted with ElGamal
- How are these ciphertexts computed?
  - By voters?
  - By election organizer's code?
  - By externally designed code?
- Mitigation: offer auditing options

Helios: IACR elections

Helios Voting Booth [exit]

IACR 2018 election

(1) Select | **(2) Review** | (3) Submit

Review your Ballot

Audit [optional]

Question #1: Director
✓ Michel Abdalla
✓ Nadia Heninger
✓ Anna Lysyanskaya
[edit responses]

Your ballot tracker is `mLmKyrdPdxiZdTIKTgJLqlVdVrxfZDIrx3NqJ4E3SRA`, and you can print it.

Once you click "Submit", the unencrypted version of your ballot will be destroyed, and only the encrypted version will remain. The encrypted version will be submitted to the Helios server.

Submit this Vote!

Election Fingerprint: `Q2gj8sqN6u+Q9dXyFRRaLtuRU6aUb4ogjNwix0GMVjU` help!

Helios: IACR elections

Helios Voting Booth [exit]

IACR 2018 election

(1) Select | **(2) Review** | (3) Submit

Review your Ballot

Question #1: Director
✓ Michel Abdalla
✓ Nadia Heninger
✓ Anna Lysyanskaya
[edit responses]

Your ballot tracker is `HKePLmAy4rL2YjuxXWMEXAiJUlj7J3IMftFA9WOMa6g`, and you can print it.

Once you click "Submit", the unencrypted version of your ballot will be destroyed, and only the encrypted version will remain. The encrypted version will be submitted to the Helios server.

Submit this Vote!

Audit [optional]

If you choose, you can audit your ballot and reveal how your choices were encrypted.

You will then be guided to re-encrypt your choices for final casting.

Verify Encryption

Election Fingerprint: `Q2gj8sqN6u+Q9dXyFRRaLtuRU6aUb4ogjNwix0GMVjU` help!

Helios: IACR elections

Helios Voting Booth [exit]

IACR 2018 election

(1) Select | **(2) Review** | (3) Submit

Your audited ballot

IMPORTANT: this ballot, now that it has been audited, *will not be tallied*.
To cast a ballot, you must click the "Back to Voting" button below, re-encrypt it, and choose "cast" instead of "audit."

Why? Helios prevents you from auditing and casting the same ballot to provide you with some protection against coercion.

Now what? Select your ballot audit info, copy it to your clipboard, then use the ballot verifier to verify it.
Once you're satisfied, click the "back to voting" button to re-encrypt and cast your ballot.


```
{"answers": [{"choices": [{"alpha":
"360456685692878682365494234104715372564036003717163237516796201779132034367166 9
462799992955411408602827640685279211682896154234321078947653022846406796638187 25
666284840458281896333923079738099335355573851703870925548920419204925839314951 27
370747843170786875019281254190609774049149178181098869330399954748519736304186 13
414269309681488826876443283985366463416552305536739767080103227250429626242008 57
332674074758205366914805658866868835690711481374430346567257931618383934376636 58
052826259748388052401523345983451290122637519512915131070585979245228549316017 53
126932073205441186453038711156881016484112062913796729290", "beta":
"134760701010083899684358350915565600632261956559226945603711075606935569293562 2
```


Before going back to voting,
you can post this audited ballot to the Helios tracking center so that others might double-check the verification of this ballot.

Even if you post your audited ballot, you must go back to voting and choose "cast" if you want your vote to count.

back to voting | post audited ballot to tracking center

Election Fingerprint: `Q2gj8sqN6u+Q9dXyFRRaLtuRU6aUb4ogjNwix0GMVjU` help!

Ballot preparation vs casting

- Votes encrypted by Helios software
- Voters can choose to cast or audit their vote
  - Audit: voters receive randomness and can use independent software to verify vote and ballot tracker
  - Cast: randomness is discarded, voters authenticate and submit their vote
- "Trust no one for integrity, trust Helios for privacy"

Helios 1.0: coercion

- Remember coercion resistance: cannot prove your vote (hence there is less incentive to buy it)
- Some protection offered by private voting booths
- How to prevent coercion when voters are at home?
- Helios v1 did not solve this, and only recommended when there is a low coercion threat (not for general elections!)
- "Coerce me!" button revealing ElGamal randomness, there for educational purposes

Coercion resistance in online voting

- How to prevent coercion when voters are at home?
- Mitigation: allow voters to change their vote multiple times until the end of election period
  (later versions of Helios, Estonian voting system)

End-to-end verifiability

- Cast as intended (individually)
  - Get confidence by using auditing procedure
- Recorded as cast (individually)
  - Check bulletin board for hash value
- Tallied as recorded (by anyone)
  - Check mixnet proof
  - Check decryption proofs

Helios 2.0

- Ben Adida, Olivier Pereira, Olivier de Marneffe
- UCLouvain 2009 president election (CP was there)
- "Weighted election"
  - Staff vote given more weight than student vote
  - End result should be revealed but not per category
- Same as above sketch, plus a proof that sum of votes is 1
- Exponential ElGamal encryption, encryption of weighted result computed with homomorphic properties

$$Enc(2v + v') = Enc(v) \otimes Enc(v) \otimes Enc(v')$$

- Decryption key shared by a set of trustees (not treshold)
- No more "Coerce me!" button (confusing to voters?)
- Same auditing capacities
- Still only recommended for low coercion threat elections

End-to-end verifiability

- Cast as intended (individually)
  - Get confidence by using auditing procedure
- Recorded as cast (individually)
  - Check bulletin board for hash value
- Tallied as recorded (by anyone)
  - Check all proofs

A non cryptographic attack on Helios

Exploiting the Client Vulnerabilities in Internet E-voting Systems: Hacking Helios 2.0 as an Example

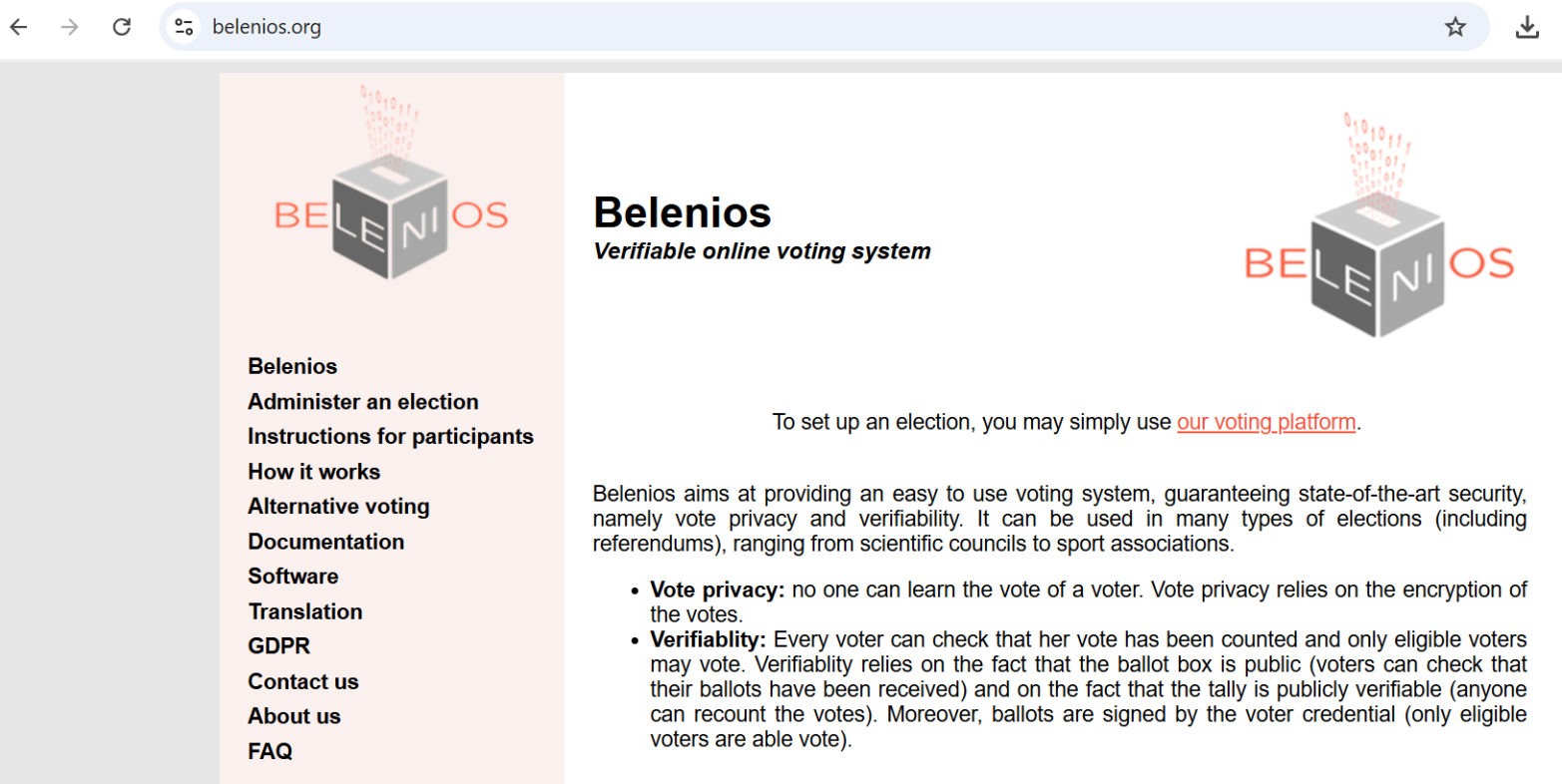
Saghar Estehghari
University College London
s.estehghari@cs.ucl.ac.uk

Yvo Desmedt
UCL and RCIS

. . . we demonstrate a simple attack against Helios 2.0 that takes advantage of the fact that every candidate in Helios can provide a URL referring to his/her candidacy statement. A malicious candidate, who wishes to win a Helios-managed election, uploads a specially crafted PDF file containing a candidacy statement to his/her website. The attack is then triggered against each voter who is using a vulnerable machine. The security of the machine is undermined, e.g., when the voter visits the attacker's webpage. In essence, we exploit Adobe Acrobat Reader's vulnerabilities to install a malicious browser extension on the voters' machines. Such an extension provides an opportunity for an attacker which may fool the voter (using Social Engineering) into accepting a hacked ballot. Due to our attack Helios 2.0 was upgraded to Helios 3.0. . . .

Belenios

"evolution of Helios that additionally provides eligibility verifiability"



Other "cryptographic" voting systems

- Scantegrity, Pret-A-Voter, STARVote, D-DEMOS ...
- Paper ballots and receipts of various formats including punch cards, bar codes, QR codes, scratch cards...
- More or less complex protocols, sometimes involving interactive proofs

Voting systems: general challenges

- Humans are bad at using crypto protocols
  - Elections do not occur often enough for routine
  - Cannot compute on their own, so need to rely on external code
  - In an interactive proof, they might not notice a swap of two messages (allowing a simulated proof)
  - Not many people will follow audit procedures
- Various randomness elements used could be trapdoored
- See Halderman, *Practical attacks on real-world e-voting*

Outline

Course organization

Introduction to e-voting

Conclusion

Conclusion

- e-voting is a nice application of cool cryptography tools (homomorphic encryption, zero-knowledge proofs, . . .)
- Complex protocols involving several entities;
  complex security model with a priori conflicting goals
- See next chapters for introduction to basic tools!
- Real world implementations offer further challenges

Lecture content

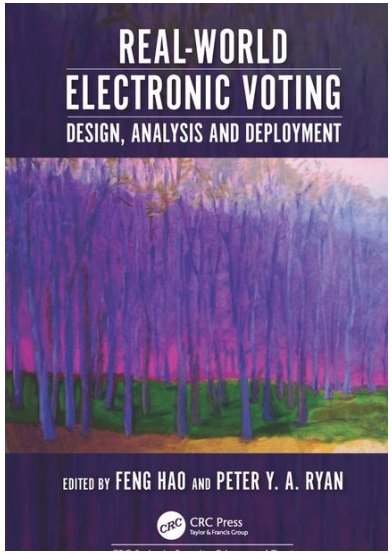
Week	Topic
1	Introduction to the course and e-voting
2	"Provable security"
3	Homomorphic encryption
4	Zero-knowledge proofs
5	Cryptanalysis
6	Post-quantum cryptography
7	Differential fault analysis
8	Security in AI
9	Security/Crypto in industry (Part 1)
10	Security/Crypto in industry (Part 2)
11	Threat model
12	Secure software implementation

Further references (1)

- B Adida, *Helios: Web-based Open-Audit Voting*, USENIX 2008
- B Adida, O de Marneffe, O Pereira, JJ Quisquater, *Electing a University President using open-audit voting: analysis of real-world use of Helios*, USENIX 2009
- S Estehghari, Y Desmedt, *Exploiting the client vulnerabilities in internet e-voting systems: hacking Helios 2.0 as an example*, USENIX 2010
- C Karlof, N Sastry, D Wagner, *Cryptographic voting protocols: a system perspective*, USENIX 2005

Further references (2)

- F Hao, P Ryan, *Real world electronic voting: design, analysis and deployment*, CRC Press



- Chapter 7: A Halderman, *Practical attacks on real-world e-voting*
- Chapter 8: S Ali, J Murray, *An overview of end-to-end verifiable voting systems*